

학생 성적 계산기

1114조건회

조건문

```
char calculateGrade(double score) {  
    if (score >= 90.0) {  
        return 'A';  
    } else if (score >= 80.0) {  
        return 'B';  
    } else if (score >= 70.0) {  
        return 'C';  
    } else if (score >= 60.0) {  
        return 'D';  
    } else {  
        return 'E';  
    }  
}
```

입력된 성적에 따라서 A,B,C,D,E를 판별하여 return합니다

반복문

```
// 1. 학생 정보 입력 및 계산 부분에 포인터 적용  
for (int i = 0; i < studentCount; i++) {  
    Student *s = &list[i]; // 현재 학생의 주소를 포인터에 저장  
  
    printf("[%d번째 학생]\n", i + 1);  
    printf("이름: ");  
    scanf("%s", s->name); // '.' 대신 '-' 사용  
  
    int sum = 0;  
    for (int j = 0; j < subjectCount; j++) {  
        while (1) {  
            printf("%s 점수 입력: ", subjects[j]);  
            scanf("%d", &s->scores[j]);  
            if (s->scores[j] >= 0 && s->scores[j] <= 100) {  
                break;  
            }  
            printf("[오류] 점수는 0점 이상 100점 이하여야 합니다. 다시 입력해 주세요.\n");  
        }  
        s->grades[j] = calculateGrade((double)s->scores[j]);  
        sum += s->scores[j];  
    }  
    s->average = (double)sum / subjectCount;  
    s->averageGrade = calculateGrade(s->average);  
    printf("\n");  
}
```

학생 수만큼 도는 바깥쪽 for문과 과목 수만큼 도는 안쪽 for문이 결합된 중첩 루프 구조입니다.
그 안에서 while(1) 무한 루프를 사용해 올바른 점수(0~100점)가 입력될 때까지 입력을 반복시키는 예외 처리를 수행합니다.

함수

```
char calculateGrade(double score);
```

```
char calculateGrade(double score) {  
    if (score >= 90.0) {  
        return 'A';  
    } else if (score >= 80.0) {  
        return 'B';  
    } else if (score >= 70.0) {  
        return 'C';  
    } else if (score >= 60.0) {  
        return 'D';  
    } else {  
        return 'E';  
    }  
}
```

입력된 성적에 따라서 A,B,C,D,E를 판별하여 return합니다

배열 & 구조체

```
typedef struct {  
    char name[20];  
    int scores[MAX_SUBJECTS];  
    char grades[MAX_SUBJECTS];  
    double average;  
    char averageGrade;  
} Student;
```

```
char calculateGrade(double score);
```

```
int main() {  
    char subjects[MAX_SUBJECTS][20];  
    int subjectCount;  
    int studentCount;
```

이름, 점수, 평균 등 자료형이 다른 관련 데이터를
'학생 1명'의 단위로 묶어 관리하기 위해 사용합니다.

Student라는 하나의 새로운 자료형을 만듦으로써,
대규모 학생 데이터를 하나의 배열로 묶어 효율적으로 통제할 수 있습니다.
실제 코드에서는 점(.)이나 화살표(→) 연산자를 사용해
각 학생의 상자 속에 성적 정보를 차곡차곡 채워 넣는 방식으로 씁니다.

구조체

```
printf("--- 결과 ---\n");
printf("%-10s", "이름");
for (int j = 0; j < subjectCount; j++) {
    printf(" | %-10s", subjects[j]);
}
printf(" | %-10s\n", "평균");

// 2. 결과 출력 부분에 포인터 적용
for (int i = 0; i < studentCount; i++) {
    Student *s = &list[i]; // 출력할 학생의 주소를 포인터에 저장

    printf("%-10s", s->name);
    for (int j = 0; j < subjectCount; j++) {
        printf(" | %3d점 (%c)", s->scores[j], s->grades[j]);
    }
    printf(" | %5.1f점 (%c)\n", s->average, s->averageGrade);
}
```

* (포인터 선언)은 실제 데이터가 들어있는 메모리 주소를 가리키는 '주소 저장용 변수'를 만들기 위해 쓰였습니다.

무거운 구조체 데이터를 통째로 복사해 오면 메모리와 시간이 낭비되기 때문에, 원본 데이터의 위치만 콕 집어서 직접 접근하고 효율적으로 수정하려고 사용한 것입니다.

전체 코드

```
1 #include <stdio.h>
2
3 #define MAX_STUDENTS 100
4 #define MAX_SUBJECTS 10
5
6 typedef struct {
7     char name[20];
8     int scores[MAX_SUBJECTS];
9     char grades[MAX_SUBJECTS];
10    double average;
11    char averageGrade;
12 } Student;
13
14 char calculateGrade(double score);
15
16 int main() {
17     char subjects[MAX_SUBJECTS][20];
18     int subjectCount;
19     int studentCount;
20
21     printf("과목 수 입력 (최대 %d): ", MAX_SUBJECTS);
22     scanf("%d", &subjectCount);
23     if (subjectCount > MAX_SUBJECTS) {
24         subjectCount = MAX_SUBJECTS;
25     }
26     if (subjectCount <= 0) {
27         printf("잘못된 과목 수입니다.\n");
28         return 1;
29     }
30
31     for (int i = 0; i < subjectCount; i++) {
32         printf("%d번째 과목명 입력: ", i + 1);
33         scanf("%s", subjects[i]);
34     }
35     printf("\n");
36
37     printf("학생 수 입력 (최대 %d): ", MAX_STUDENTS);
38     scanf("%d", &studentCount);
39     if (studentCount > MAX_STUDENTS) {
40         studentCount = MAX_STUDENTS;
41     }
42     if (studentCount <= 0) {
43         printf("잘못된 학생 수입니다.\n");
44         return 1;
45     }
46     printf("\n");
47
48     Student list[MAX_STUDENTS];
49 }
```

```
// 1. 학생 정보 입력 및 계산 부분에 포인터 적용
for (int i = 0; i < studentCount; i++) {
    Student *s = &list[i]; // 현재 학생의 주소를 포인터에 저장

    printf("[%d번째 학생]\n", i + 1);
    printf("이름: ");
    scanf("%s", s->name); // '.' 대신 '-' 사용

    int sum = 0;
    for (int j = 0; j < subjectCount; j++) {
        while (1) {
            printf("%s 점수 입력: ", subjects[j]);
            scanf("%d", &s->scores[j]);
            if (s->scores[j] >= 0 && s->scores[j] <= 100) {
                break;
            }
            printf("[오류] 점수는 0점 이상 100점 이하여야 합니다. 다시 입력해 주세요.\n");
        }
        s->grades[j] = calculateGrade((double)s->scores[j]);
        sum += s->scores[j];
    }

    s->average = (double)sum / subjectCount;
    s->averageGrade = calculateGrade(s->average);
    printf("\n");
}

printf("--- 결과 ---\n");
printf("%-10s", "이름");
for (int j = 0; j < subjectCount; j++) {
    printf(" | %-10s", subjects[j]);
}
printf(" | %-10s\n", "평균");

// 2. 결과 출력 부분에 포인터 적용
for (int i = 0; i < studentCount; i++) {
    Student *s = &list[i]; // 출력할 학생의 주소를 포인터에 저장

    printf("%-10s", s->name);
    for (int j = 0; j < subjectCount; j++) {
        printf(" | %3d점 (%c)", s->scores[j], s->grades[j]);
    }
    printf(" | %5.1f점 (%c)\n", s->average, s->averageGrade);
}

return 0;
}
```

```
96
97 char calculateGrade(double score) {
98     if (score >= 90.0) {
99         return 'A';
100    } else if (score >= 80.0) {
101        return 'B';
102    } else if (score >= 70.0) {
103        return 'C';
104    } else if (score >= 60.0) {
105        return 'D';
106    } else {
107        return 'E';
108    }
109 }
```


실행결과

과목 수 입력 (최대 10): 3
1번째 과목명 입력: 수학
2번째 과목명 입력: 과학
3번째 과목명 입력: 영어

학생 수 입력 (최대 100): 3

[1번째 학생]
이름: 조건희
수학 점수 입력: 60
과학 점수 입력: 70
영어 점수 입력: 80

[2번째 학생]
이름: 박준호
수학 점수 입력: 90
과학 점수 입력: 50
영어 점수 입력: 67

[3번째 학생]
이름: 윤건호
수학 점수 입력: 91
과학 점수 입력: 76
영어 점수 입력: 34

--- 결과 ---

이름	수학	과학	영어	평균
조건희	60점 (D)	70점 (C)	80점 (B)	70.0점 (C)
박준호	90점 (A)	50점 (E)	67점 (D)	69.0점 (D)
윤건호	91점 (A)	76점 (C)	34점 (E)	67.0점 (D)

감사합니다